#stac2017

GebとSpockではじめるシステムテスト自動化

2017/12/10 システムテスト自動化カンファレンス2017-2

大中浩行(@setoazusa)

この作品は クリエイティブ・コモンズ 表示 4.0 国際 ライセンスの下に提供されています。

Copyright 2017 Hiroyuki Onaka

みなさまへのお願い

- 講演の様子は撮影可です。
- シャッター音が出ないようにしてください。
- このスライドはslideshareで公開します
- 感想はハッシュタグ #stac2017 まで

Copyright 2017 Hiroyuki Onaka

アジェンダ(1)

Groovyで書かれたWebDriver拡張であるGebと、BDDスタイルのテスティングフレームワークであるSpockによるシステムテストの自動化について、その特徴をプロジェクトの採用を通じて得られた知見を含めながら紹介します。

Copyright 2017 Hiroyuki Onaka

アジェンダ(2)

- テスト自動化ピラミッド
- Gebによるシステムテスト自動化
- Gebの建前と本音
- まとめ

Copyright 2017 Hiroyuki Onaka

わたしについて

- TDD Boot Camp(TDDBC)方面
- SIerの傭兵部隊所属
- アプリケーションエンジニア(主にJava)とインフラエンジニアの二刀流
  - 自動デプロイとかBlue-Green Deploymentとかやってたらいつのまにかこういうことになった
- 技術系同人サークル「ふぃーるどの一つ」主宰
  - 国会図書館で「TDDの発展史」とか「Geb Spock」とかで検索すると結果に出てくる

Copyright 2017 Hiroyuki Onaka

本スライドのサンプルコードは、

https://github.com/azusa/stac2017-geb

で公開しています。

Copyright 2017 Hiroyuki Onaka

テスト自動化ピラミッド

Copyright 2017 Hiroyuki Onaka

…その前に

「システムテスト」とは？

Copyright 2017 Hiroyuki Onaka

よくある分類

- 単体テスト
- 結合テスト
- システムテスト

Copyright 2017 Hiroyuki Onaka

この言葉の呼び方って定義が拡散しますよね

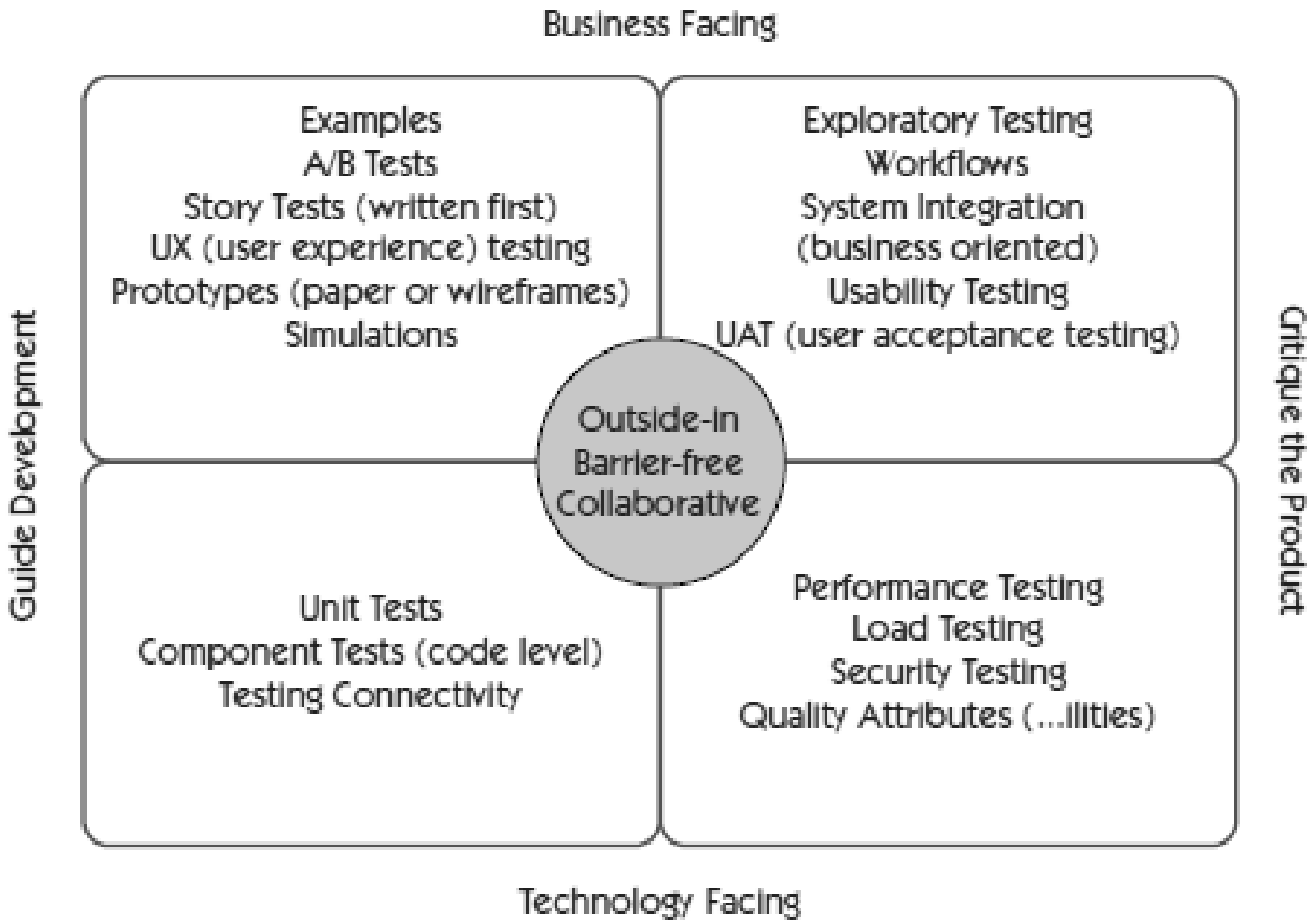
「画面が関係するものは結合テストで扱います」

「単体テストではブラウザーから画面を叩いて、デバッガーで挙動を確認します」

Copyright 2017 Hiroyuki Onaka

Gregory, Janet, and Lisa Crispin.
「More Agile Testting Learining Journeys for the Whole Team.」

新・アジャイルテストの四象限



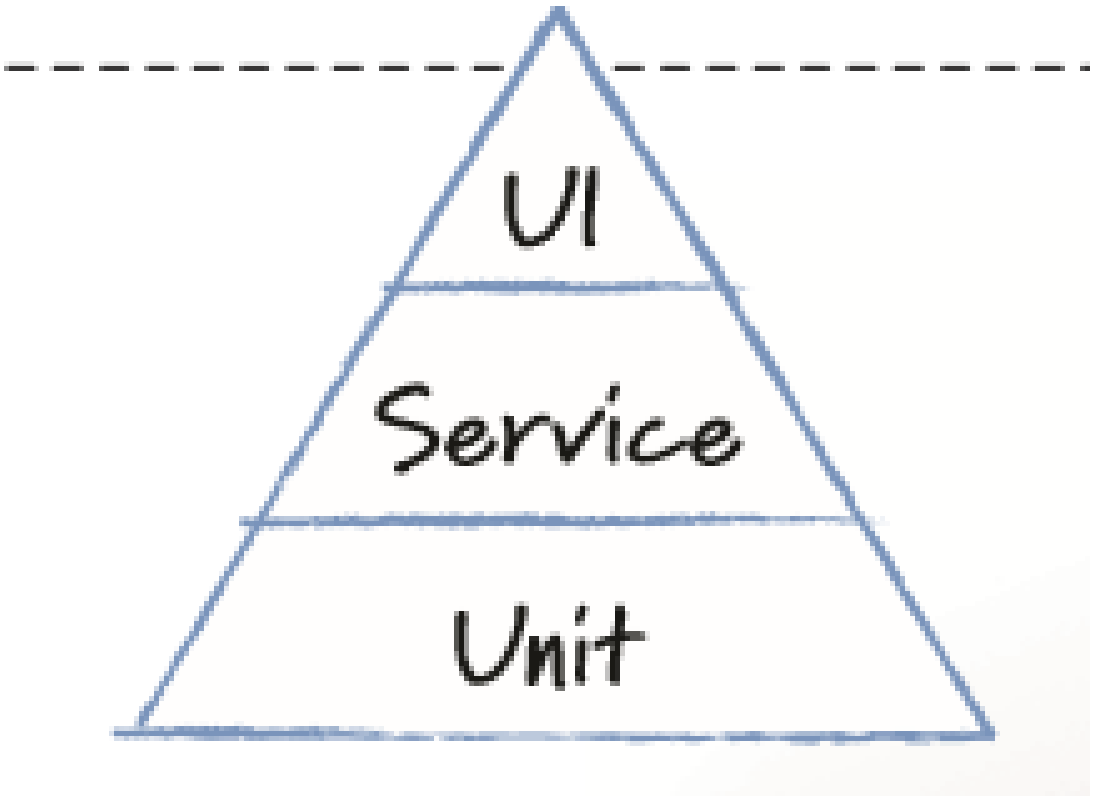
Copyright 2017 Hiroyuki Onaka

Googleのテスト分類

「グーグルでは、形態よりも規模を協調するために、コード、統合、システムテストという区別をせず、Sテスト、Mテスト、Lテストという表現を使う」

Copyright 2017 Hiroyuki Onaka

「テスト自動化ピラミッド」



Copyright 2017 Hiroyuki Onaka

The Addison-Wesley Signature Series

A MIKE COHN SIGNATURE BOOK

Succeeding with Agile

Software Development Using Scrum

- テスト自動化のレイヤーを「UI」「Service」「Unit」の3つに分類したもの。
- 「Service」に対するテストが、アプリケーションのインターフェスに対するテストを、UI(ユーザーインターフェース)を迂回して実行することが特徴。

Copyright 2017 Hiroyuki Onaka

ここで使う「システムテスト」

UI(ユーザーインターフェース)を通して、エンドツーエンドでシステムの動作を確認する

Copyright 2017 Hiroyuki Onaka

エンドツーエンドのつらさ

- データのセットアップつらい…
- 実行時間つらい…
- コード何もいじってないのにテスト落ちた、つらい…

Copyright 2017 Hiroyuki Onaka

Sam Newman(著) 佐藤直生(監訳)
「マイクロサービスアーキテクチャ」

O'REILLY
オライリー・ジャパン

マイクロサービスアーキテクチャ

エンドツーエンドテストの難易度の増大。

- システム相互の疎結合化、分散化
- それに伴う非同期処理の増大

結局昔やってた「結合一発勝負」とかわらなくなってくる

Copyright 2017 Hiroyuki Onaka

「ストーリーでなくジャーニーをテストする」

これに対抗する最善の方法は、少数の中核となるジャーニーに焦点を絞ってシステム全体をテストする方法です。この中核となるジャーニーで対象になっていない機能は、互いに分離してサービスを分析するテストで対処する必要があります。このジャーニーは相互に合意され、共同で所有される必要があります。音楽専門店の例では、CD の注文、商品の返品、新規顧客の作成といった（高価値な対話であり極めて少数の）動作に焦点を絞るでしょう。

Copyright 2017 Hiroyuki Onaka

ところで、「ジャーニー」ってなによ

正直よくわからない

おそらく重要なのは、テストの粒度ではなく「相互に合意され、共同で所有される必要」のほう

Copyright 2017 Hiroyuki Onaka

相互に合意することが必要な理由

継続的に開発が進むサービスにおいて、「システムテストの粒度を粗くする」ということは

- プロダクトオーナー(企画)サイドがリスクをどこまで許容できるか
- 監視アラートへの立ち上がりを迅速に出来るか

という、ITサービス運用のリスク管理の視点と関係してくるため

Copyright 2017 Hiroyuki Onaka

一つだけわかっていること

システムテストを自動化することは、「ユーザーストーリーをエンドツーエンドで網羅する自動テストを作成する」ことではない

また、「メンテナンス対象となるシステムが一個増える」くらいの覚悟が必要

Copyright 2017 Hiroyuki Onaka

#stac2017

Gebによるシステムテスト自動化

Copyright 2017 Hiroyuki Onaka

Geb

- GroovyによるSelenium WebDriver拡張
- http://gebish.org/
- ライセンス: Apache License Version 2.0

Copyright 2017 Hiroyuki Onaka

Gebの特徴

- Groovy の言語機能を生かした簡潔な記述
- jQuery ライクなDOM へアクセスするための式言語
- ページオブジェクトのサポート
- 任意のテスティングフレームワークと組み合わせて使用(Spock/Junit/TestNG/Cucumeber-JVM)

Copyright 2017 Hiroyuki Onaka

Spock

- Groovyで動作するテスティングフレームワーク(BDDスタイル)
- Groovyの特徴を生かしたテスト記述
  - PowerAssert/データ駆動/DSLによるテストケース記述 etc
- Spock信者

Copyright 2017 Hiroyuki Onaka

Gebを採用した背景

- エンドツーエンドテストの自動化を検討したが、キャプチャーアンドリプレイだと以下の理由で立ちゆかなくなるのが目に見えてた(2014年当時)

Copyright 2017 Hiroyuki Onaka

Gebを採用した背景(1/3)

キャプチャーアンドリプレイでは規模の拡大にともない保守が苦しくなる

Copyright 2017 Hiroyuki Onaka

Gebを採用した背景(2/3)

アジャイル開発プロセスの導入に伴い、イテレーションの進行につれてリグレッションテストの工数が拡大していくのに備える必要があった

Copyright 2017 Hiroyuki Onaka

Gebを採用した背景(3/3)

シングルページアプリケーション(Backbone.js)導入に伴い、画面遷移が非同期が基本になるため、キャプチャーアンドリプレイでは画面遷移のところに個別にwaitを追加する必要が出てくる

Copyright 2017 Hiroyuki Onaka

Gebを採用したのは、自分の周囲にGroovyをやっている人が多かったから

Copyright 2017 Hiroyuki Onaka

Spockを採用した背景

Geb+JUnitでプロトタイプ作成してチームメンバーに渡したらいつの間にかGeb+Spockになっていた

(実話)

Copyright 2017 Hiroyuki Onaka

https://github.com/SeleniumHQ/selenium/wiki/PageObjects

ページオブジェクト

public メソッドは、ページが提供するサービスを表す

- ページの内部構造を露出しないようつとめる
- 原則として(ページオブジェクト内で) アサーションを行わない
- メソッドは他のページオブジェクトを返す
- 一つのページ全体を(一つの) ページオブジェクトで表す必要は無い
- 同じ操作が異なる結果となる場合は、異なるメソッドとしてモデル化する

Copyright 2017 Hiroyuki Onaka

Gebによるページオブジェクト

```
class GebishOrgHomePage extends Page {

    static at = { title == "Geb - Very Groovy Browser Automation" }

    static content = {
        manualsMenu { module(ManualsMenuModule) }
    }
}
```

Copyright 2017 Hiroyuki Onaka

Gebによるテスト(Spock+ページオブジェクト)

```
def "can get to the current Book of Geb"() {
    when:
    to GebishOrgHomePage
    and:
    manualsMenu.open()
    then:
    manualsMenu.links[0].text().startsWith("current")
    when:
    manualsMenu.links[0].click()
    then:
    at TheBookOfGebPage
}
```

Copyright 2017 Hiroyuki Onaka

Gebによるテスト(JUnit+ページオブジェクト未使用)

#stac2017

```
@Test
 void canGetToTheCurrentBookOfGeb() {
     go "http://gebish.org/"
     $("div.menu a.manuals").click()
     waitFor { ! $("#manuals-menu").hasClass("animating")}
     assert  $("#manuals-menu").find("a")[0].text().startsWith("current")
     $("#manuals-menu").find("a")[0].click()
     waitFor { title.startsWith("The Book Of Geb") }
 }
```

Copyright 2017 Hiroyuki Onaka

Gebによるテストの組み合わせ

- ページオブジェクト使う - 使わない
- テスティングフレームワーク
  - Spock - JUnit - TestNG - Cucumber-JVM
- Maven - Gradle

Copyright 2017 Hiroyuki Onaka

テストのスタイルについてどう選択をするか

結論は、

ページオブジェクト+Spock+Gradle

一択

Copyright 2017 Hiroyuki Onaka

なぜページオブジェクトなのか

各ページのDOM要素にid属性を付けて要素の特定に使う、という技法がSelenium WebDriverのテストにはあります。

Copyright 2017 Hiroyuki Onaka

htmlの要素にテストのためのid属性をつけることの問題

#stac2017

- SPA(シングルページアプリケーション)だとスコープがアプリケーション全体になる
- アプリケーションからの動的なDOM生成が増えると、idやclass属性を通してアプリケーションのロジックとテストが密結合してしまう

「フロントエンドをリファクタリングしたらエンドツーエンドのテストが全滅」とか...

Copyright 2017 Hiroyuki Onaka

https://qiita.com/akameco/items/519f7e4d5442b2a9d2da

「idやclassを使ってテストを書くのは、もはやアンチパターンである」

#stac2017

Qiita ホーム コミュニティ キーワードを入力 ストック一覧 投稿する 28

220

@akameco 2017年11月22日に更新

idやclassを使ってテストを書くのは、もはやアンチパターンである

test e2e jest React enzyme

いきなり結論を書くと、`id`や`class`はスタイルのためのものなので、テストでそれを使うのはやめましょう。そして、カスタムデータ属性を使いましょう。(`id`や`class`はスタイルのためだけではないという意見はごもっともです！しかし、主にとしてスタイルに使われるということでご了承頂いて以下の駄文に付き合って頂けると幸いです🙇)

先に断っておくと主にreactについての話で、JSXを前提とします。(手法はReactに限りませんが理由は後述)

はじめに

ご存知の通り、ロジックとスタイルは別物です。
もしスタイルの変更のためにclass名を変えて、ロジックのテストが落ちるなんてことはあってはならないのです。

はじめに
修正困難なクラス名
data-test属性
プロダクションビルドではカスタムデータ属性を取り除く
誰が使っているか？
まとめ
参考
ツイッターやはてブのコメントの反映
😉 こういう動機に至らせてしまうReactがそもそも開発プラットフォームとしては微妙な気がしている
😉 idやclassはスタイルのためだけではない
😉 うちはjs用のclass名の命名規則があるので、デザインを変えてもそのclass名は変わらない
😇 テスト書いてない
+α デモ

現実的な最適解

- セレクターでDOM要素への指定を記述して、ページオブジェクトでそれをラップする。
  - Gebの場合はjQueryライクなセレクターを使用可能
- テストからはページオブジェクト経由でアクセスすることで、DOMの要素を隠蔽し、htmlの構造がかわったときの影響範囲を限定する

Copyright 2017 Hiroyuki Onaka

GebによるテストでSpockを選択する理由

Gebによるテスト記述はストーリーベースの記述となるため、BDDスタイルのSpockでの記述が向いている。

後パラメーター駆動のテストもやりやすい

Copyright 2017 Hiroyuki Onaka

もう一つのSpockを選択する理由:spock-reports

Report for GebishOrgSpec

Summary:

Created on Sat Dec 09 17:36:20 JST 2017 by azusa

Executed features	Failures	Errors	Skipped	Success rate	Time
1	0	0	0	100.0%	14.611 seconds

Features:

- Book of Gebの最新バージョンが表示できる

Book of Gebの最新バージョンが表示できる

Return

When: Gebのホームページを表示する
And: マニュアルのメニューを開く
Then: currentのリンクがcurrentではじまっている
When: リンクをクリックする
Then: The Book Of Gebのページが表示される

Generated by Athaydes Spock Reports

Copyright 2017 Hiroyuki Onaka

Gradleを選択する理由

ビルドに関するワークフローが単純なものには

Mavenが向いているが、

Copyright 2017 Hiroyuki Onaka

Gebによるテストには、スクリプトとしての細かい制御が求められる

- クロスブラウザー
- テスト環境の切替
- テストスイートの管理など

よってGradle

Copyright 2017 Hiroyuki Onaka

Gebの建前と本音

Copyright 2017 Hiroyuki Onaka

- 非同期処理
- スクリーンショット
- クロスブラウザー
- GebとGroovy

Copyright 2017 Hiroyuki Onaka

非同期処理(建前)

Geb では非同期処理の記述を行うには、要素の出現判定する処理のブロックをwaitFor{ 条件}で囲みます。

タイムアウトまでの時間は、GebConfig.groovy の waiting ブロックのクロージャーでtimeout プロパティーを設定します。

Copyright 2017 Hiroyuki Onaka

非同期処理(本音)

DOMの出現でいくらWaitかけたってサーバー側の非同期処理はWaitのしようがない

不安定なGebのシナリオのかなりの割合が、サーバー側の非同期処理に対するWaitが適切でないことによります

Copyright 2017 Hiroyuki Onaka

サーバー側の非同期処理

データ連携などの非同期処理の結果を参照できるAPIを用意するなど、アーキテクチャーで手当をすべきです。

それがないと、データベースをポーリングしてデータが反映されるのを待つとか、sleepの秒数で調節するとか、つらいことになります。

Copyright 2017 Hiroyuki Onaka

スクリーンショット(建前)

- geb.spock.GebReportingSpec を継承することで、テストの実行時に自動でスクリーンショットを取得することができます。

Copyright 2017 Hiroyuki Onaka

スクリーンショット(本音)

- トーストの消える消えない等のアニメーションの動きを、スクリーンショットを見てわかるには超能力が必要
- SPAにおける、アプリケーションのハングアップもまた爾り

Copyright 2017 Hiroyuki Onaka

画面を録画するソリューション

- BrowserStackとかSauce Labsとか
- 画面を録画する用途にしては大げさすぎやしませんか？

Copyright 2017 Hiroyuki Onaka

というわけで、OSSだけで画面録画を目指してみましょう

用意するもの

- Ubuntu(17.10)
- ブラウザー(Firefox)
- Xvfb
- tmux
- FFmpeg

Copyright 2017 Hiroyuki Onaka

```
Xvfb :1  -screen 0 1920x1200x24 -ac &

tmux new-session -d -s Recording1 'ffmpeg -y  -f
x11grab -video_size 1920x1200 -i :1 -codec:v
libx264 -r 12 -ab 128k /vagrant/out4.mp4'

export DISPLAY=:1

./gradlew test -Pbrowser=firefox

tmux send-keys -t Recording1 q
```

Copyright 2017 Hiroyuki Onaka

やっていること

- Xvfbで起動した仮想デスクトップをFFmpegでキャプチャー
- Gradleによるテストを、仮想デスクトップ上で実行
- FFmpegは終了するのに「q」のキーシーケンスを送る必要があるので、tmuxのセッション上で起動

Copyright 2017 Hiroyuki Onaka

詳しくはブログで

http://blog.fieldnotes.jp/entry/recordiing-selenium-test

Copyright 2017 Hiroyuki Onaka

クロスブラウザー(建前)

GebConfig.groovy 内のdriver という変数に

- WebDriver の実装クラス名を指す文字列
- WebDriver のインスタンスを返すクロージャー

を示すことで、使用するブラウザーを切り替えることができます。

Copyright 2017 Hiroyuki Onaka

クロスブラウザー(本音)

- 1ブラウザーでも不安定なテスト安定させるの大変なのにクロスブラウザーとか正直やってられない
- 大体ここでやりたいことはユーザーインターフェースの単体テストではない

Copyright 2017 Hiroyuki Onaka

geb/geb-example.gradle
https://github.com/geb/geb-example-gradle

それでもクロスブラウザーをやるというなら

Geb では、実行時にシステムプロパティー geb.env を設定することにより、設定ファイル (GebConfig.groovy) のenvironmentsブロックで設定値の切り替えを行うことができます。

Gebの公式サンプルでは、この機能を使ってブラウザーの切替を行っています。

Copyright 2017 Hiroyuki Onaka

ですが

ブラウザーの切り替えにgeb.envを使うと、対象の環境(開発環境とかステージング環境とか)の切替をどこに持たせるかという問題がおきます。

またGebの公式サンプルでは、IDEからの実行を考慮していないという問題

Copyright 2017 Hiroyuki Onaka

そうすると、起動時にフラグでブラウザーを指定して、GebConfig.groovyの中で切替やセットアップをすることになるわけですが...

Copyright 2017 Hiroyuki Onaka

画像はイメージです(1)

```
switch (targetBrowser) {

    case "chrome" :
        driver = { new ChromeDriver() }
        setUpChromeDriver(BUILD_DIR)
        break
    case "firefox" :
        atCheckWaiting = 1
        def geckodriverFilename = SystemUtils.IS_OS_WINDOWS ? "geckodriver.exe" : "geckodriver"
        System.setProperty "webdriver.gecko.driver", "$BUILD_DIR/webdriver/geckodriver/$geckodriverFilename
        driver = { new FirefoxDriver() }
        break
    case "chromeHeadless" :
        setUpChromeDriver(BUILD_DIR)
        driver = {
            ChromeOptions o = new ChromeOptions()
            o.addArguments( ...arguments: 'headless')
            new ChromeDriver(o)
        }
        break
    case "edge" :
        def edgedriverFile = new File( pathname: "MicrosoftWebDriver.exe")
        System.setProperty "webdriver.edge.driver", edgedriverFile.absolutePath
        driver = { new EdgeDriver()}
        break
```

画像はイメージです(2)

```
task downloadChromeDriver {
    def outputFile = file("$buildDir/webdriver/chromedriver.zip")
    inputs.property("chromeDriverVersion", chromeDriverVersion)
    outputs.file(outputFile)

    doLast {
        def driverOsFilenamePart
        if (Os.isFamily(Os.FAMILY_WINDOWS)) {
            driverOsFilenamePart = "win32"
        } else if (Os.isFamily(Os.FAMILY_MAC)) {
            driverOsFilenamePart = "mac64"
        } else if (Os.isFamily(Os.FAMILY_UNIX)) {
            driverOsFilenamePart = Os.isArch("amd64") ? "linux64" : "linux32"
        }
        FileUtils.copyURLToFile(new URL("http://chromedriver.storage.googleapis.com/${chromeDriverVersion}/chrom
    }
}

task unzipChromeDriver(type: Copy) {
    def outputDir = file("$buildDir/webdriver/chromedriver")
    dependsOn downloadChromeDriver
    outputs.dir(outputDir)

    from(zipTree(downloadChromeDriver.outputs.files.singleFile))
    into(outputDir)
}
```

このやり方はちとつらい

というわけで今回紹介するのが、

ブラウザーごとにWebDriverのセットアップを自動でやってくれるWebDriverManager (https://github.com/bonigarcia/webdrivermanager)

です。

Copyright 2017 Hiroyuki Onaka

先ほどのコードがこうなります

```
switch (targetBrowser) {

    case "chrome" :
        ChromeDriverManager.getInstance().setup()
        driver = "org.openqa.selenium.chrome.ChromeDriver"
        break
    case "firefox" :
        atCheckWaiting = 1
        FirefoxDriverManager.getInstance().setup();
        driver = "org.openqa.selenium.firefox.FirefoxDriver"
        break
```

Copyright 2017 Hiroyuki Onaka

GebとGroovy(建前)

Geb は、Groovy の言語機能を生かした簡潔な記述でテストを書くことができます。

Copyright 2017 Hiroyuki Onaka

GebとGroovy(本音)

IDEの補完効かない

Copyright 2017 Hiroyuki Onaka

それに対する解: Intellij IDEA

```
then: "The Book Of Gebのページが表示される"
at The
```

- TheBookOfGebPage (default package)
- Theories (org.junit.experimental.theories)
- Theory (org.junit.experimental.theories)
- ThemeReader (sun.awt.windows)
- ThenBlock (org.spockframework.compiler.model)
- ThemeClient (com.sun.webkit)
- ThemeClientImpl (com.sun.javafx.webkit)
- TheoryAnchor (org.junit.experimental.theories.Theories)
- ThrowablesEvent (jdk.jfr.events)
- ThrowErrorManager (jdk.nashorn.internal.runtime.Context)

Ctrl+下 and Ctrl+上 will move caret down and up in the editor >>

Copyright 2017 Hiroyuki Onaka

まとめ

Copyright 2017 Hiroyuki Onaka

システムテストを自動化するということ

システムテストを自動化するということは、

「メンテナンスするシステムが１個増える」

くらいに考えたほうがいい

Copyright 2017 Hiroyuki Onaka

それでもなぜ自動化するのか

- 自動化のゴールをコスト削減に向けると、全体として縮小均衡にしかならない
  - 私は「テスト自動化によってテスト要員を○名削減できました」という仕事がしたいわけではない

Copyright 2017 Hiroyuki Onaka

自動化によって「何を減らせるか」ではなく、自動化によって「新しいことが出来るようになる」、ということに目を向けるべき

Copyright 2017 Hiroyuki Onaka

「システムテストを自動化する」ことによって新しくできることになることは？

#stac2017

その前に、一個エピソードを紹介したいと思います。

Copyright 2017 Hiroyuki Onaka

大手通信会社の法人向けサービスの事例

開発チームがGebで作成したエンドツーエンドのテストを、Javaのバージョンアップや、Blue-Green Deployment導入に伴うインフラ更改の際の動作確認として使用した、という事例。

Copyright 2017 Hiroyuki Onaka

システムテストが自動化されていたことにより、インフラがサービス水準の改善のために積極的にインフラの構成に手をいれることができるようになった、ということ。

Copyright 2017 Hiroyuki Onaka

「システムテストを自動化する」ことによって新しくできることになることは？

#stac2017

複数の異なる粒度のテストのレイヤーが自動化されて組み合わさることにより、

- 継続的なサービスな改善に対応できるテスト工程が構築されること。
- 手動テストが、「魅力的品質」寄りのテストに注力できるようになること。

Copyright 2017 Hiroyuki Onaka

ありがとうございました！

- 大中浩行(Onaka,Hiroyuki)
- @setoazusa
- グロースエクスパートナーズ株式会社
  アーキテクチャソリューション部
  テクニカルリード
- http://hiroyuki.fieldnotes.jp/

Copyright 2017 Hiroyuki Onaka